青 岛 科 技 大 学

高等学历继续教育

本 科 毕 业 设 计 （论 文）

题 目 基于 Flask+Bootstrap 的企业官网 CMS 的设计与实现

指导教师	XXX
专业名称	XXX
层　　次	专升本
学生姓名	XXX
学生学号	XXX

2025 年 5

基于 Flask+Bootstrap 的企业官网 CMS 的设计与实现

摘 要

互联网技术快速发展，企业数字化转型不断深入。在此背景下，企业官方网站已成为品牌形象展示、产品信息发布和客户在线沟通的重要平台。市面上 WordPress 和 Drupal 等主流 CMS 体量较大，定制也不容易。功能需求相对明确的中小型企业如果采用这些系统，往往会面临学习成本偏高、维护负担偏重的问题。基于上述情况，本文设计并实现了一套基于 Flask 微框架和 Bootstrap 5 前端框架的轻量级企业官网内容管理系统。后端用 Python 语言开发，按 MVC 架构模式分层。Flask 蓝图机制将业务逻辑划分为前台展示、用户认证和后台管理三个独立模块。数据持久化层使用 SQLAlchemy ORM 操作 SQLite 数据库，建了 roles、users、articles、categories、pages、media_assets、site_settings、contact_messages 共 8 张表。前端页面用 Bootstrap 5 栅格系统做响应式布局，Jinja2 模板引擎在服务端完成页面渲染，桌面端与移动端均可正常访问。安全机制上，Flask-Login 负责用户会话管理，Werkzeug 库处理密码哈希存储，Flask-WTF 提供 CSRF 跨站请求伪造防护。同时按角色对后台功能菜单做了权限区分。前台面向访客，提供首页展示、栏目浏览、文章详情、站内搜索和联系留言等功能。后台面向管理人员，提供仪表盘数据概览、文章全生命周期管理、栏目层级维护、媒体文件上传、用户账号管理和站点全局配置等功能。测试阶段对前台展示与后台管理共 15 个核心测试用例分别做了功能测试和性能测

试。从测试情况看，各模块功能正常，数据处理未出现错误，页面响应较快。整体上能够满足中小型企业官网内容管理的日常使用。

关键词：Flask 框架；Bootstrap；内容管理系统；企业官网；响应式设计；角色权限控制

前言

企业官网是企业在互联网上的对外窗口，需要完成品牌展示、产品推介、资讯发布、客户沟通等工作。市场竞争加剧后，企业对官网的内容更新频率和页面展示质量都有了更高的期望。传统的静态 HTML 网站需要专业技术人员手动修改代码才能完成内容变更维护成本高、更新周期长，难以适应企业频繁发布新闻动态、调整产品介绍和回应客户咨询的实际运营节奏。内容管理系统的出现在一定程度上缓解了这一矛盾，使不具备编程技能的运营人员也能通过可视化后台完成文章发布、栏目调整、图片更换和联系方式修改等日常维护工作，降低了企业网站日常运营的技术门槛。

不过 WordPress 和 Drupal 虽然功能丰富、社区成熟，但系统本身比较重，插件依赖关系复杂，安全补丁更新也比较频繁。对功能需求明确、技术团队有限的中小型企业来说，学习成本、定制难度和运维工作量都不小。在中小型企业官网这个场景下，做一套架构简洁、功能聚焦、容易改和部署的轻量级 CMS，仍有实际的工程应用价值。

本项目选用 Python 语言的 Flask 微框架作为后端核心开发工具，结合 Bootstrap 5 前端框架实现跨终端的响应式界面设计。Flask 的特点是轻量灵活、扩展自由，在 Python Web 开发社区中使用较多。开发时可以根据项目需要逐个引入数据库、认证、表单处理等扩展，不用一次性装进整个全栈框架的所有组件。系统整体遵循 MVC 架构模式进行分层组织，通过 SQLAlchemy ORM 框架实现数据持久化操作，使用 Flask-Login 管理用户会话状态，借助 Flask-WTF 完成表单数据校验和 CSRF 安全防护，并采用基于角色的访问控制策略实

现后台功能的权限隔离。后续章节分别从需求分析、架构与数据库设计、前后台功能实现、系统测试几个方面，记录该企业官网 CMS 的设计和开发过程。

1 绪论

1.1 课题背景与意义

企业数字化转型推进以来，官方网站在品牌形象展示、产品信息发布和客户互动沟通方面的作用越来越明显。市场竞争加剧之后，企业对网站内容的更新频率和管理效率的要求也在提高。传统的静态网页改内容要手动修改 HTML 代码，效率不高，也容易出错。企业日常运营中对内容频繁更新、动态展示、多角色协作管理的需求，静态网页很难支撑。有一套功能完善、操作方便的内容管理系统，对企业日常的网站运营工作会有比较直接的帮助。

目前市场上主流的内容管理系统如 WordPress、Drupal 等均基于 PHP 语言开发，虽然功能强大且插件生态丰富，但存在系统臃肿、定制化困难、学习曲线陡峭等问题，对于中小型企业而言往往超出实际需求。Python 的生态系统近几年日趋成熟，在 Web 开发领域的使用范围也在扩大。Flask 是 Python 生态中的一个微型 Web 框架，体量小、扩展灵活，比较适合构建功能需求明确的轻量级内容管理系统。

Bootstrap 是目前使用较多的前端响应式框架，内置了界面组件和栅格布局系统，拿来搭多终端兼容的页面比较快。Flask 负责后端逻辑，Bootstrap 负责前端页面，两者搭配可以做出前后端一体的全栈方案。本项目要做的是一套架构清晰、功能实用的企业官网 CMS，包含文章管理、栏目管理、单页管理、媒体管理、用户管理、留言管理、站点设置和全文搜索这些模块，给中小型企业提供一种轻量、易维护的内容管理方案。

1.2 国内外研究现状

国际上，内容管理系统已经发展了二十多年，产品体系比较成熟。WordPress 的全球市场份额超过百分之四十，插件和主题数量庞大，各类网站场景都能覆盖。Drupal 在内容架构设计和安全更新方面做得比较扎实，政府和大型企业网站用得较多。Joomla 介于两者之间，功能和上手难度都处于中间水平。这两年无头 CMS 的概念开始流行。Strapi、Contentful 这类产品只负责内容的增删改查和 API 输出，前端用什么框架由开发者自行决定。

在国内，内容管理系统的发展经历了从 DedeCMS、PHPCMS 等早期 PHP 系统到现代化框架的演进过程。Python 在国内的使用范围不断扩大，基于 Python 的 Web 框架也逐步进入开发者视野。Django 作为 Python 生态中功能最全的 Web 框架，提供了内置的 Admin 后台和 ORM 系统，适合快速搭建管理系统。Flask 生态系统中的 Flask-Admin、Flask-Security 等扩展库为构建管理系统提供了丰富的工具支持。不过目前国内用 Python 做的轻量级企业 CMS 还比较少，多数开源项目停留在个人博客层面，面向企业官网场景的完整方案不多见。

前端技术上，响应式设计目前已经是企业建站的基本配置。Bootstrap 的栅格系统和 UI 组件库比较成熟，企业 Web 应用中用得很多。Vue、React 这些前端框架推动了前后端分离架构的流行。不过对内容展示为主的企业官网来说，服务端渲染配合 Bootstrap 的方案在 SEO 和开发效率上还是有一些优势的。Jinja2 模板引擎和 Bootstrap 一起用，页面开

发可以利用模板继承和组件复用，代码可维护性和开发效率都能兼顾。

现有的 CMS 产品在功能上已经比较完善，但用 Python Flask 框架做的轻量级企业 CMS 方案还不多。本项目的出发点就是这个空白，用 Flask 框架来做一套企业 CMS，争取在架构清晰、功能完整和部署简便之间找到平衡点。

1.3 研究目标

本文的研究目标是设计并实现一套基于 Flask 和 Bootstrap 的企业官网内容管理系统，为中小型企业提供一种轻量化、功能完整的网站内容管理解决方案。系统需要满足以下几项具体目标。功能层面，要覆盖文章管理、栏目管理、单页管理、媒体管理、用户管理、留言管理、站点设置和全文搜索等模块，对应企业官网日常内容运营的主要操作。架构层面，采用 MVC 模式分层设计，用 Flask 蓝图做模块拆分，保持代码结构清晰、后续扩展方便。

安全层面，按角色区分管理员和编辑者的操作权限，避免越权操作。展示层面，前台页面用 Bootstrap 做响应式布局，桌面端和移动端均可正常访问。操作层面，后台界面采用 B/S 架构，操作入口尽量直观，让不懂技术的运营人员也能完成日常内容更新。

1.4 论文组织结构

全文共分七章，各章内容安排如下：

第 1 章为绪论，介绍课题背景与意义、国内外研究现状和研究目标，说明论文的整体组织结构。

第 2 章为关键技术介绍，分别对 Flask 框架、Bootstrap 前端框架、SQLAlchemy 与 SQLite、Jinja2 模板引擎、Flask-Login 与 Flask-WTF 等开发技术做说明。

第 3 章为系统需求分析，从技术、经济和操作三个方面做可行性论证，明确前台和后台的功能需求，提出安全性、性能、兼容性和可维护性等非功能需求。

第 4 章为系统设计，包含功能设计、架构设计、业务流程设计和数据库设计四部分，给出系统功能结构图、架构图、业务流程图和 E-R 图。

第 5 章为系统功能的实现，分前台和后台两部分展示各模块的实现效果和关键代码。

第 6 章为系统测试，说明测试方法和测试用例，展示功能测试结果并进行分析。

第 7 章为总结与展望，总结项目工作，分析不足并提出改进方向。

2 关键技术介绍

2.1 Flask 框架

Flask 由 Armin Ronacher 开发，是一个 Python 微型 Web 框架，核心依赖 Werkzeug WSGI 工具包和 Jinja2 模板引擎。所谓微型，不是说功能少，而是框架核心只保留必要部分，其他功能按需扩展。和 Django 这种大而全的框架不同，Flask 把数据库、表单验证、身份认证这些组件的选择权留给了开发者，项目需要什么就装什么。Flask 通过装饰器实现 URL 路由映射，提供了封装完善的请求与响应对象，支持模块化设计的 Blueprint 蓝图机制，以及推荐用于大型项目的应用工厂模式。本项目按功能将系统拆分为 main、auth 和 admin 三个蓝图，分别负责前台展示、后台登录和后台管理功能，各蓝图拥有独立的路由前缀和模板目录，便于维护和扩展。

2.2 Bootstrap 前端框架

Bootstrap 由 Twitter 团队开发并开源，目前已经到了第五个大版本。Bootstrap 5 的响应式栅格布局基于十二列网格和五个断点等级，分别对应小屏手机、平板、中等屏幕、大屏幕和超大屏幕。不同设备访问时页面会自动调整列数和间距。预构建的组件有导航栏、卡片、表单、模态框、分页器、警告提示等，后台管理和前台展示两边的界面需求基本都能覆盖。Bootstrap 5 把 jQuery 依赖去掉了，交互功能改用原生 JavaScript，前端资源体积减小了不少。本项目还引入了 Bootstrap Icons 图标库，后台和前台页面都用这套图标，视觉风格保持统一。

2.3 SQLAlchemy 与 SQLite

SQLAlchemy 是 Python 生态中使用较多的 ORM 工具。它把 Python 类映射为数据库表，将对象操作转换为 SQL 语句，开发者可以用面向对象的写法来操作数据库。它支持 SQLite、MySQL、PostgreSQL 等多种数据库后端，对外提供 ORM 层和 Core 层两种操作接口。Flask-SQLAlchemy 是 Flask 与 SQLAlchemy 的集成扩展，配置流程和会话管理都做了简化。开发者在配置文件中写好数据库连接字符串，就可以用模型基类定义数据模型，会话的创建和回收由扩展自动处理。

本项目开发阶段用的是 SQLite。SQLite 是一款无服务器、零配置、单文件存储的嵌入式数据库引擎，对于中小规模的内容管理系统而言，其性能完全能够满足适度并发访问的需求。由于 SQLAlchemy 的数据库抽象层设计，当系统需要部署到生产环境时，可以便捷地切换到 MySQL 或 PostgreSQL 等更强大的数据库系统，无需修改业务代码。

2.4 Jinja2 模板引擎

Jinja2 是 Flask 默认的模板引擎，在 Python Web 开发中使用面很广。它支持模板继承、宏定义、过滤器、自动转义等特性。自动转义功能在输出变量时会自动对 HTML 特殊字符做编码，防止跨站脚本攻击。Jinja2 用双花括号表示变量输出，花括号百分号表示控制流语句，语法简洁直观。模板继承是 Jinja2 比较核心的特性。定义一个 base.html 基础模板，里面设好可覆写的 block 区块，各页面的子模板只需要填充各自不同的部分，公共 HTML 不用重复写。同时通过 include 指令可以将导航栏、侧边栏、页脚等公共组件抽取

为独立片段进行复用。Flask 还有上下文处理器的机制，可以向所有模板注入全局变量。比如站点名称、导航菜单数据这些每个页面都要用的内容，在上下文处理器里设置一次即可，不用每个视图函数单独传。

2.5 Flask-Login 与 Flask-WTF

Flask-Login 是 Flask 生态中专门用于用户会话管理的扩展库，它提供了 UserMixin 基类用于简化用户模型的实现，login_required 装饰器用于保护需要登录的视图函数，同时支持记住登录和匿名用户处理等功能。Flask-Login 与 Flask 的会话 Cookie 机制无缝集成，在用户登录后自动维护会话状态，在请求处理前自动加载当前用户对象，为系统的身份认证提供了完整的解决方案。本项目在此基础上实现了基于角色的访问控制，通过自定义装饰器检查当前用户的角色权限，区分管理员和编辑者可访问的功能范围。

Flask-WTF 将 Flask 和 WTForms 表单库做了整合，主要处理表单提交和验证的工作。它的几个主要功能点是 CSRF 防护、表单字段验证和文件上传处理。WTForms 用类属性的方式定义表单字段和验证规则，比如必填、长度限制、邮箱格式检查等，表单逻辑和视图逻辑各自独立。另外，Flask 底层依赖的 Werkzeug 工具包提供了 generate_password_hash 和 check_password_hash 两个函数，用于对密码做加盐哈希处理。存入数据库的是哈希值而非明文，即使数据库泄露也无法直接还原密码。

2.6 本章小结

本章对系统开发所用的关键技术做了介绍。Flask 作为后端框架负责路由分发和业务

逻辑组织，Bootstrap 5 提供响应式前端页面布局，SQLAlchemy 配合 SQLite 完成数据持久化，Jinja2 负责服务端模板渲染，Flask-Login 和 Flask-WTF 分别处理用户会话管理和表单安全校验。上述技术的选用为后续系统设计与实现提供了技术支撑。

3 系统需求分析

3.1 可行性分析

3.1.1 技术可行性

本系统后端选用 Flask 框架，Flask 是 Python 生态中使用最广泛的微型 Web 框架之一，基于 Werkzeug WSGI 工具包和 Jinja2 模板引擎构建，自 2010 年发布以来已经历了十多年的迭代和社区验证，API 接口稳定、文档完善、第三方扩展库丰富。前端选用 Bootstrap 5 框架，Bootstrap 是全球使用量最大的 CSS 框架，其栅格系统和预构建组件可以快速搭建响应式页面，GitHub 上 Star 数量超过 17 万，技术成熟度毋庸置疑。数据库选用 SQLite 配合 SQLAlchemy ORM，SQLAlchemy 是 Python 语言中最成熟的对象关系映射工具，支持多种数据库后端切换。整体技术方案各个组件均为成熟的开源项目，拥有活跃的社区支持和大量的使用案例，技术风险低。

3.1.2 经济可行性

本项目使用的所有框架和工具均为开源免费软件，不涉及任何商业授权费用。Flask 遵循 BSD 协议，Bootstrap 遵循 MIT 协议，SQLite 属于公共领域软件，SQLAlchemy 遵循 MIT 协议。开发工具方面，VS Code 编辑器免费使用，Python 运行环境免费获取。数据库方面，SQLite 作为嵌入式数据库引擎，不需要独立的数据库服务器进程，整个数据库以单个文件形式存储在应用目录中，省去了数据库服务器的采购和维护成本。硬件方面，开发和测试在普通笔记本电脑上即可完成，部署时 Flask 应用和静态资源可以放在同一台

入门级云服务器上运行。综合来看，系统的开发成本和后续运维成本均很低，经济可行性良好。

3.1.3 操作可行性

系统基于 B/S 架构，用户在浏览器中即可访问前台页面和管理后台，不需要安装任何客户端软件。前台页面基于 Bootstrap 实现响应式布局，在桌面端、平板和手机上均能获得良好的浏览体验，访客可以通过常见的导航栏、栏目列表和搜索框等交互方式获取所需信息，学习成本很低。后台管理界面使用左侧菜单加右侧内容区的布局，功能分类清晰，文章编辑、栏目维护、媒体上传等操作流程直观明了，管理员和编辑人员不需要专业的技术培训就能上手操作。系统部署和更新也较为简单，只需替换服务端文件并重启应用即可完成版本升级。

3.2 功能需求分析

企业官网 CMS 要解决的核心问题是：前台展示和后台维护放在同一套系统里，中小型企业的运营人员可以统一管理。根据企业官网的常见运营场景，系统角色分为管理员和访客两种。管理员在后台维护内容和管理系统，访客在前台浏览企业信息、提交留言。管理员用例图如图 3-1 所示，访客用例图如图 3-2 所示。

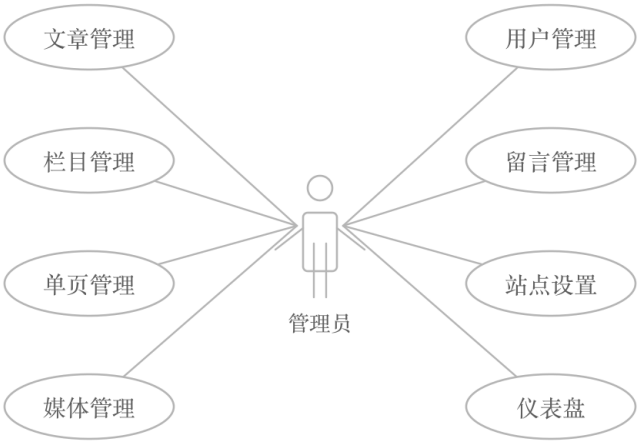

图 3-1 管理员用例图

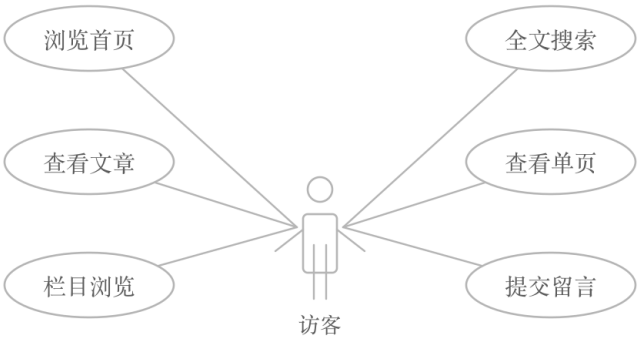

图 3-2 访客用例图

3.2.1 前台功能需求

前台面向普通访客，主要提供信息浏览和互动功能。首页展示企业推荐文章和栏目

入口，访客可按栏目分类浏览文章列表，点击进入文章详情页阅读完整内容。站内搜索功能支持按关键词检索文章标题和内容。单页功能用于展示关于我们、服务介绍等固定页面。联系留言功能允许访客填写姓名、邮箱和留言内容提交给管理员。整个前台浏览流程无需登录。

3.2.2 后台功能需求

后台面向管理人员，提供内容维护和系统配置功能。文章管理支持新增、编辑、删除、状态切换（草稿/发布/归档）和推荐置顶。栏目管理支持层级结构配置和导航显示控制。单页管理维护固定页面内容。媒体资源管理负责图片上传和文件信息记录。用户管理维护后台账号和角色分配。留言管理处理访客提交的联系信息。站点设置维护站点名称、联系方式等全局参数。仪表盘提供文章数、用户数、留言数等数据概览。

3.3 非功能需求分析

安全性方面，系统需要防范常见的 Web 安全威胁。密码不能明文存储，必须做哈希处理。表单提交需要 CSRF 令牌防护。后台页面必须经过登录验证和角色权限检查才能访问。

性能方面，前台页面加载时间应控制在合理范围内，后台管理操作的响应也不能太慢。数据库查询要建好索引，避免全表扫描。

兼容性方面，前台页面需要兼容主流浏览器（Chrome、Firefox、Safari、Edge），同时支持桌面端和移动端的响应式显示。

可维护性方面，代码结构按模块划分，蓝图之间职责清晰。数据库层使用 ORM 抽象，方便后续切换数据库类型。模板使用继承机制复用公共布局。

3.4 本章小结

本章从可行性分析、功能需求和非功能需求三个方面对系统进行了需求分析。可行性分析从技术、经济和操作三个角度论证了方案的可行性。功能需求明确了前台访客和后台管理员两类角色的操作场景。非功能需求对安全性、性能、兼容性和可维护性提出了具体要求。以上分析为后续的系统设计提供了依据。

4 系统设计

4.1 系统功能设计

根据前期需求分析的结果，系统功能分成了前台展示模块、用户认证模块和后台管理模块三个部分，划分依据是使用角色和业务场景。前台展示模块面向普通访客，包含首页浏览、栏目导航、文章阅读、单页访问、站内搜索和联系留言六项功能。访客可以浏览企业资讯，也可以提交留言反馈。用户认证模块面向后台管理人员，负责登录验证、登出操作和角色权限校验。只有登录并且权限匹配的用户才能访问对应的后台管理功能。

后台管理模块是功能最集中的部分，下面分成文章管理、栏目管理、单页管理、媒体资源管理、用户管理、留言管理和站点设置七个子模块。文章管理负责文章的新增、编辑、删除、状态切换与推荐置顶；栏目管理支持层级结构配置和导航显示控制；单页管理用于维护关于我们等静态页面；媒体资源管理负责图片上传与元数据记录；用户管理维护后台账号信息；留言管理处理访客提交的联系信息；站点设置维护站点名称、联系方式和首页文案等全局参数。

三个模块各自独立又相互关联。前台展示模块从数据库中读取后台录入的内容来渲染页面，用户认证模块负责后台管理模块的访问安全，后台管理模块为前台展示持续提供内容数据。系统功能结构图如图 3-1 所示。

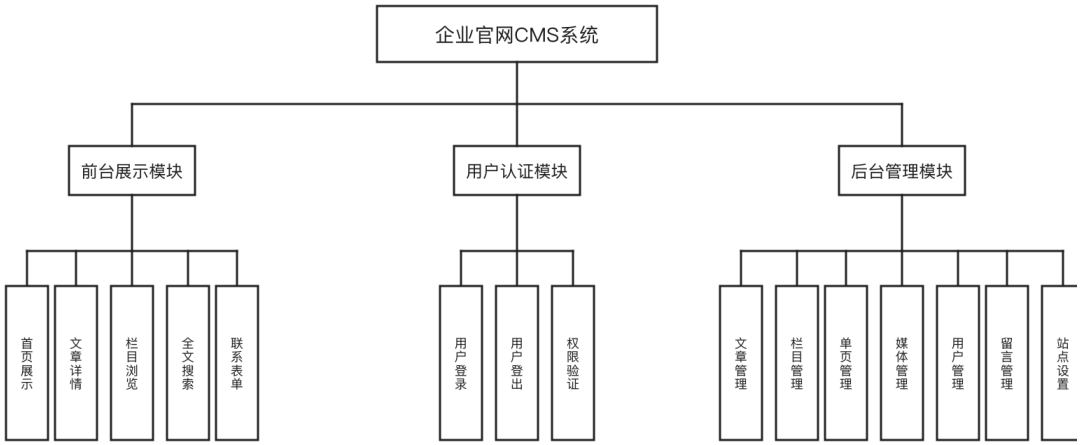

图 3-1 系统功能结构图

4.2 系统架构设计

系统基于 B/S 架构，用户在浏览器中访问前台页面与管理后台，服务器端负责业务逻辑处理和页面渲染。系统初始化时使用 Flask 应用工厂模式，代码结构按 MVC 分层思想来组织。模型层用 SQLAlchemy ORM 定义数据实体及其关联关系，视图层由 Jinja2 模板引擎完成页面渲染，控制层则由蓝图路由分发请求并组织业务流程。

模块划分上，按功能职责拆成了 main、auth 和 admin 三个蓝图。main 蓝图注册在根路径下，负责前台展示逻辑，包括首页、栏目页、文章详情页、单页、搜索页和联系页的路由与视图；auth 蓝图注册在/auth 路径下，负责后台登录与登出功能；admin 蓝图注册在/admin 路径下，负责文章、栏目、单页、媒体、留言、用户和站点设置等管理功能。每个蓝图有独立的路由前缀、视图函数和模板目录，模块边界比较清晰，维护和扩展时互

不影响。

数据访问层用 Flask-SQLAlchemy 扩展，模型类统一管理数据库表结构与外键关联，避免手工拼接 SQL 语句带来的安全隐患和维护成本。开发阶段用的 SQLite，数据库文件放在 instance 目录下。如果部署到生产环境，改一下配置里的连接字符串就可以切到 MySQL 等关系型数据库。表现层由 Jinja2 模板引擎和 Bootstrap 5 框架负责前台与后台页面构建，模板继承机制用来复用导航栏、页脚和侧边栏等公共布局组件，实现前台响应式展示和后台统一管理界面。系统总体架构图如图 3-2 所示。

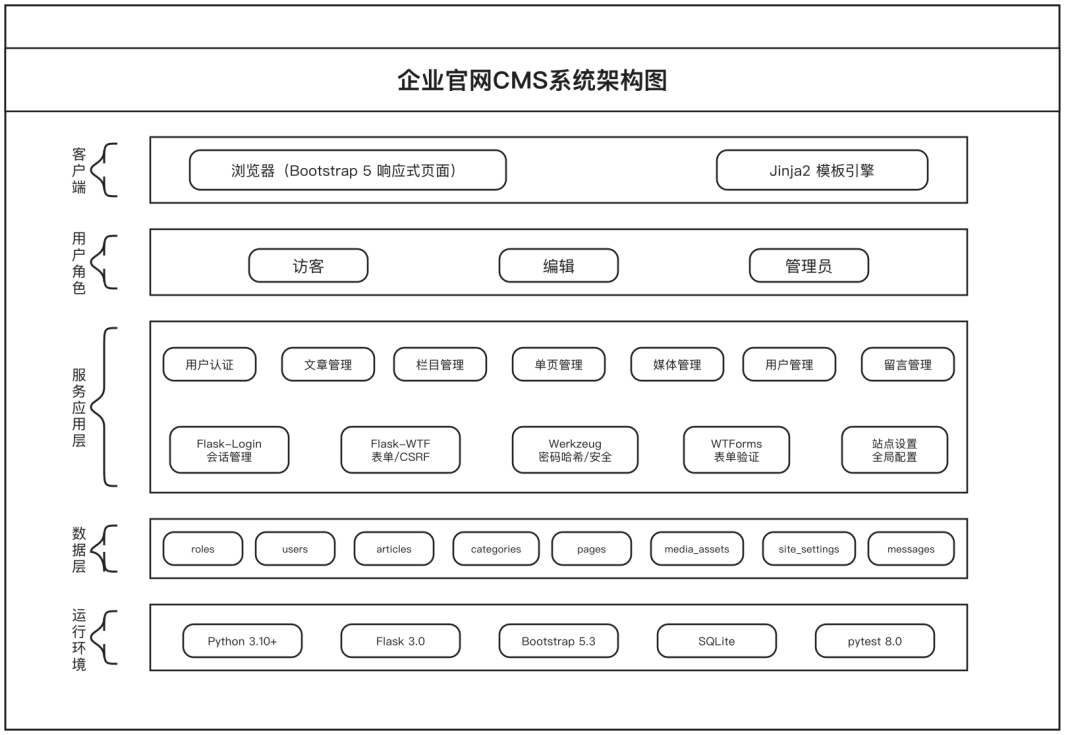

图 3-2 系统总体架构图

请求处理的大致链路是：浏览器发出 HTTP 请求，Flask 路由把请求分发到对应蓝图的视图函数。视图函数依次完成参数校验、模型调用和权限判断，将处理结果传递给 Jinja2 模板进行渲染并返回 HTML 响应。对于后台管理请求，视图函数执行前，Flask-

Login 会校验用户登录状态，自定义装饰器会校验角色权限；对于前台展示请求，则根据文章和栏目的发布状态过滤展示内容，保证未发布内容不对外可见。

4.3 业务流程设计

系统的业务流程围绕访客浏览和管理员维护两条主线展开。访客进入系统后，首页会展示推荐文章和栏目入口，点击栏目查看分类文章列表，选择具体文章阅读详情内容，也可通过搜索功能按关键词检索文章。如果访客需要联系企业，可进入联系页面填写姓名、邮箱和留言内容并提交。整个前台浏览流程无需登录，访客可自由获取公开发布的内容信息。

管理员通过后台登录页面输入用户名和密码完成身份验证后，进入后台管理界面。管理员可以进行文章发布与编辑、栏目结构维护、单页内容更新、媒体资源上传、留言查看处理、用户账号管理和站点参数配置等操作。后台所有操作均需通过登录校验和角色权限判断，编辑角色仅可访问内容管理相关功能，管理员角色拥有全部管理权限。系统业务流程图如图 3-3 所示。

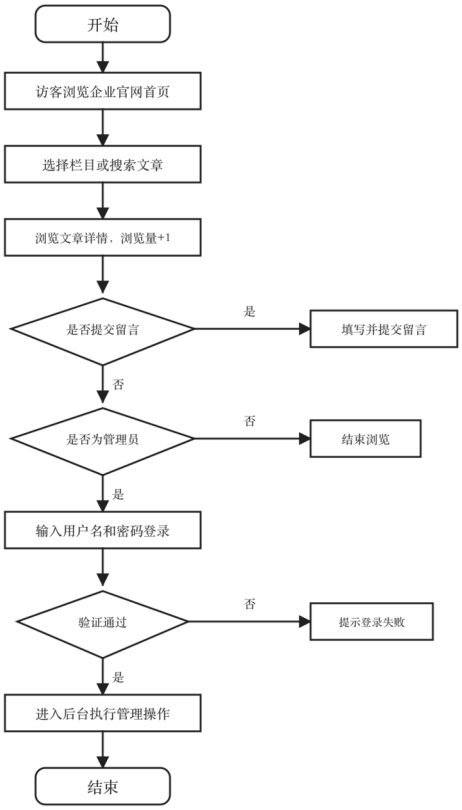

图 3-3 系统业务流程图

在内容发布流程上，管理员新建文章时设置为草稿状态，经过编辑校对后切换为已发布状态，前台页面即可展示该文章。如需临时下架，可将状态切换为已归档。这样文章从创建到发布再到归档的整个过程都通过状态字段来控制，不会出现内容状态不明的情况。

4.4 数据库设计

系统数据库设计分为概念结构设计和逻辑结构设计两个阶段，分别从实体关系建模和数据表结构定义两个层面完成数据组织方案的设计。

4.4.1 概念结构设计

根据系统功能需求，梳理出用户、文章、栏目、媒体资源、单页和联系留言六个核

心业务实体。下面分别对各实体的属性及其 E-R 图进行说明。

（1）用户实体。用户实体用于描述后台管理人员的基本信息，属性包括用户编号（主键）、用户名、邮箱、密码哈希、账号状态、最后登录时间、角色外键、创建时间和更新时间。用户实体 E-R 图如图 3-4 所示。

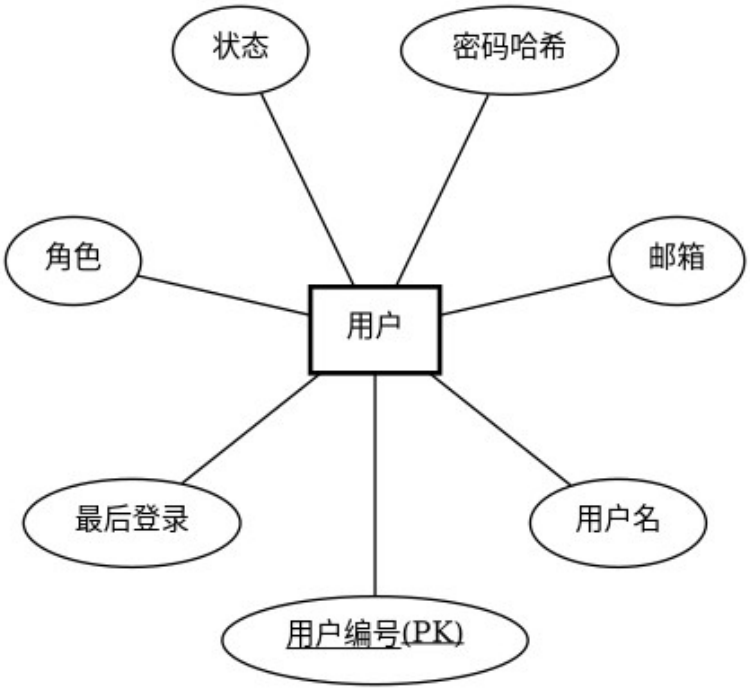

图 3-4 用户实体 E-R 图

（2）文章实体。文章实体是系统内容管理的核心数据对象，属性包括文章编号（主键）、标题、URL 别名、摘要、正文内容、封面图片、发布状态、置顶标记、推荐标记、浏览次数、发布时间、所属栏目外键和作者外键。文章实体 E-R 图如图 3-5 所示。

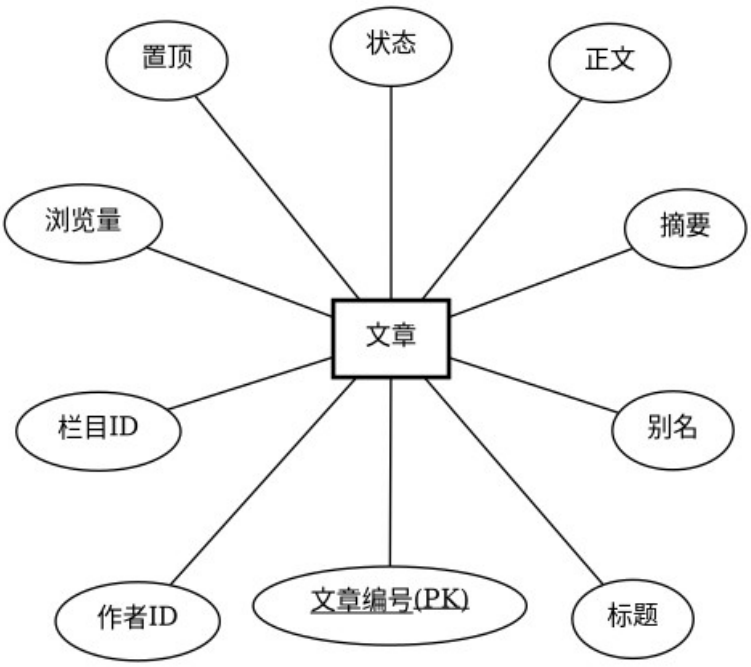

图 3-5 文章实体 E-R 图

（3）栏目实体。栏目实体用于组织文章的分类结构，属性包括栏目编号（主键）、栏目名称、URL 别名、描述信息、排序值、是否显示在导航栏、栏目状态和父栏目外键。栏目通过自引用外键实现父子层级关系。栏目实体 E-R 图如图 3-6 所示。

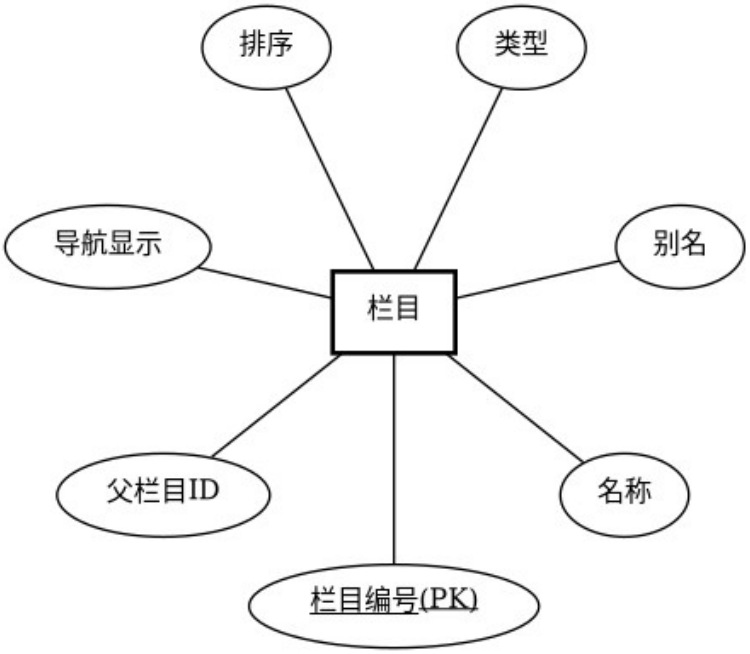

图 3-6 栏目实体 E-R 图

（4）媒体资源实体。媒体资源实体用于记录后台上传的图片等文件信息，属性包括

资源编号（主键）、原始文件名、存储文件名、MIME 类型、文件大小、上传者外键和上传时间。媒体资源实体 E-R 图如图 3-7 所示。

图 3-7 媒体资源实体 E-R 图

（5）单页实体。单页实体用于管理关于我们、服务说明等静态页面，属性包括页面编号（主键）、标题、URL 别名、正文内容、是否显示在导航栏、排序值、页面状态、创建时间和更新时间。单页实体 E-R 图如图 3-8 所示。

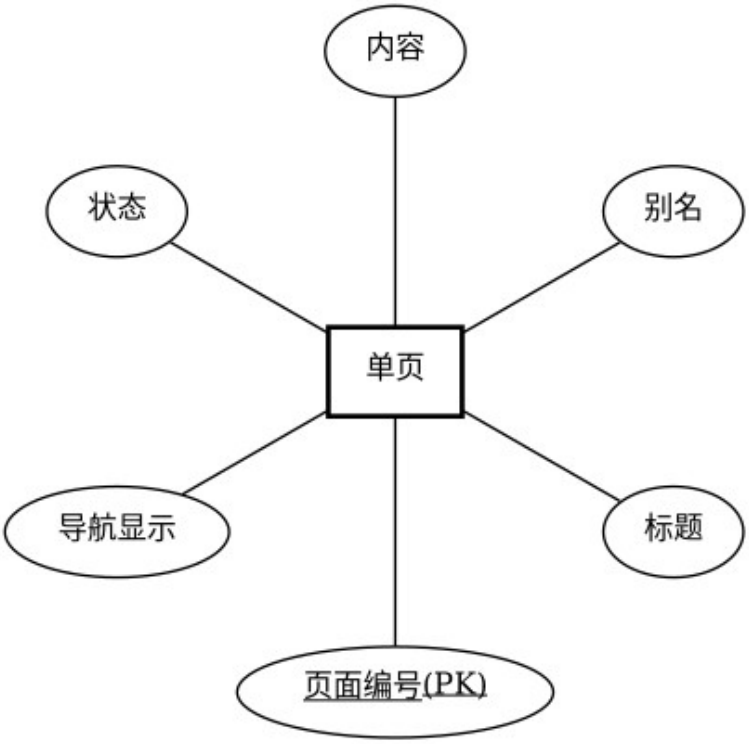

图 3-8 单页实体 E-R 图

（6）联系留言实体。联系留言实体用于保存访客通过联系页提交的反馈信息，属性包括留言编号（主键）、姓名、邮箱、电话号码、留言内容、处理状态和提交时间。联系留言实体 E-R 图如图 3-9 所示。

图 3-9 联系留言实体 E-R 图

上述六个实体之间存在如下关联关系：用户与文章为一对多关系，一个用户可发布多篇文章；栏目与文章为一对多关系，一个栏目下可包含多篇文章；栏目与栏目之间为自引用一对多关系，实现层级分类；用户与媒体资源为一对多关系，一个用户可上传多个媒体文件。系统总体 E-R 图如图 3-10 所示。

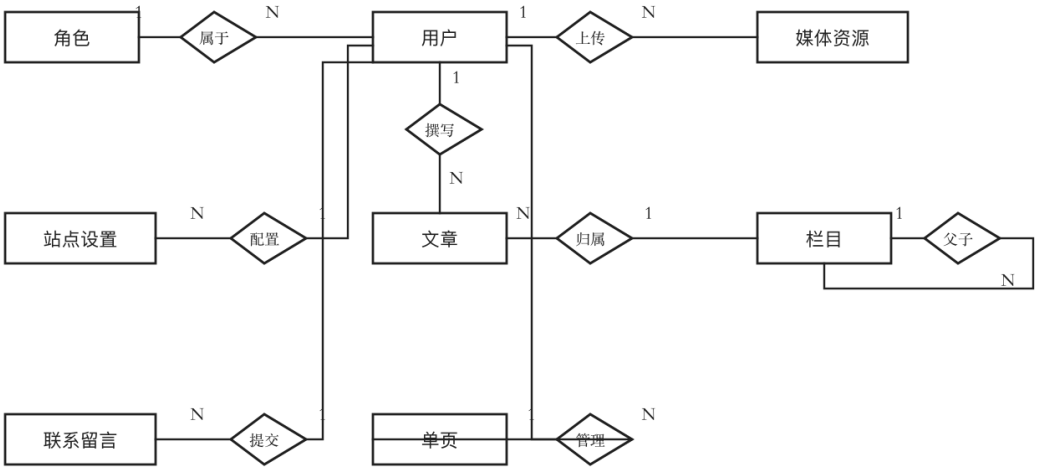

图 3-10 系统总体 E-R 图

4.4.2 逻辑结构设计

根据概念结构设计的 E-R 模型，将各实体转化为具体的数据库表结构。系统共设计 8 张数据表，下面分别列出各表的字段定义。

表 4-1 roles 角色表

字段名	数据类型	说明
id	INTEGER	主键，自增
name	VARCHAR(50)	角色名称（admin/editor）
description	VARCHAR(200)	角色描述
created_at	DATETIME	创建时间

表 4-2 users 用户表

字段名	数据类型	说明
id	INTEGER	主键，自增
username	VARCHAR(80)	用户名（唯一）
email	VARCHAR(120)	邮箱地址（唯一）
password_hash	VARCHAR(256)	密码哈希值
is_active	BOOLEAN	账号状态
last_login	DATETIME	最后登录时间
role_id	INTEGER	角色外键
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

表 4-3 categories 栏目表

字段名	数据类型	说明
id	INTEGER	主键，自增
name	VARCHAR(100)	栏目名称
slug	VARCHAR(100)	URL 别名（唯一）
description	TEXT	栏目描述
sort_order	INTEGER	排序值
show_in_nav	BOOLEAN	是否显示在导航栏
is_active	BOOLEAN	栏目状态
parent_id	INTEGER	父栏目外键（自引用）
created_at	DATETIME	创建时间

表 4-4 articles 文章表

字段名	数据类型	说明

id	INTEGER	主键，自增
title	VARCHAR(200)	文章标题
slug	VARCHAR(200)	URL 别名（唯一）
summary	TEXT	文章摘要
content	TEXT	正文内容（HTML）
cover_image	VARCHAR(500)	封面图片路径
status	VARCHAR(20)	状态（draft/published/ archived）
is_sticky	BOOLEAN	是否置顶
is_featured	BOOLEAN	是否推荐
view_count	INTEGER	浏览次数
published_at	DATETIME	发布时间
category_id	INTEGER	所属栏目外键
author_id	INTEGER	作者外键
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

表 4-5 pages 单页表

字段名	数据类型	说明
id	INTEGER	主键，自增
title	VARCHAR(200)	页面标题
slug	VARCHAR(200)	URL 别名（唯一）
content	TEXT	页面正文
show_in_nav	BOOLEAN	是否显示在导航
sort_order	INTEGER	排序值
is_active	BOOLEAN	页面状态
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

表 4-6 media_assets 媒体资源表

字段名	数据类型	说明
id	INTEGER	主键，自增
original_name	VARCHAR(255)	原始文件名
stored_name	VARCHAR(255)	存储文件名
mime_type	VARCHAR(100)	MIME 类型
file_size	INTEGER	文件大小（字节）
uploader_id	INTEGER	上传者外键
created_at	DATETIME	上传时间

表 4-7 contact_messages 联系留言表

字段名	数据类型	说明
id	INTEGER	主键，自增
name	VARCHAR(100)	留言者姓名
email	VARCHAR(120)	邮箱地址

phone	VARCHAR(20)	电话号码
message	TEXT	留言内容
is_read	BOOLEAN	是否已读
created_at	DATETIME	提交时间

表 4-8 site_settings 站点设置表

字段名	数据类型	说明
id	INTEGER	主键，自增
key	VARCHAR(100)	设置项键名（唯一）
value	TEXT	设置项值
description	VARCHAR(200)	设置项说明
updated_at	DATETIME	更新时间

上述 8 张表通过 SQLAlchemy 模型定义外键约束和索引。users 表的 username 和 email 字段设置了唯一约束，articles 表的 slug 字段设置了唯一约束以保证 URL 地址唯一，categories 表的 slug 字段同样设置了唯一约束。所有 DateTime 类型字段在模型层通过 default 参数设置默认值，确保记录创建时自动填充时间戳。

4.5 本章小结

本章分别从功能设计、架构设计、业务流程设计和数据库设计四个方面对企业官网 CMS 进行了整体设计。功能设计部分把系统拆成了前台展示、用户认证和后台管理三个模块，明确了各自的职责和功能组成。架构设计确定了基于 Flask 蓝图的 B/S 分层架构方案；业务流程设计描述了访客浏览和管理员维护两条核心业务路径；数据库设计部分用 E-R 建模和关系表定义完成了六个业务实体的数据组织。以上设计成果为下一章的功能实现提供了依据。

5 系统功能的实现

5.1 前台功能模块实现

前台功能模块由 main 蓝图承载，面向所有访客提供首页浏览、栏目导航、文章阅读站内搜索和联系留言等功能。所有前台页面继承自 base.html 基础模板，导航栏和页脚结构是统一的，页面布局用 Bootstrap 5 的栅格系统做响应式适配。

5.1.1 首页展示实现

首页由 main 蓝图的 home 路由渲染，页面包含英雄横幅区域、置顶文章区域、最新文章列表和栏目导航四个板块。置顶文章是查询 is_top 标志为真的已发布文章得到的，最新文章按发布时间倒序查询前六条记录，栏目列表按排序字段升序加载。首页运行效果如图 5-1 所示。

图 5-1 系统首页运行效果

首页路由的关键实现代码如下。

```
@bp.route('/')
def home():
    top_articles = (
        Article.query.filter_by(status='published', is_top=True)
        .order_by(Article.published_at.desc().nullslast())
        .limit(3).all()
    )
    latest_articles = (
        Article.query.filter_by(status='published')
        .order_by(Article.published_at.desc().nullslast())
```

```
    .limit(6).all()
)
categories = Category.query.filter_by(status=True)\
    .order_by(Category.sort_order.asc()).all()
return render_template('main/home.html',
    top_articles=top_articles,
    latest_articles=latest_articles,
    categories=categories)
```

5.1.2 栏目与文章浏览实现

栏目列表页对应 category_detail 路由，根据 URL 中的栏目 slug 查询对应栏目下的已发布文章并分页展示。文章详情页对应 article_detail 路由，根据文章 slug 查询文章记录，页面加载时自动将浏览量加一，并在底部展示同栏目下的相关推荐文章。栏目文章列表页面如图 5-2 所示，文章详情页面如图 5-3 所示。

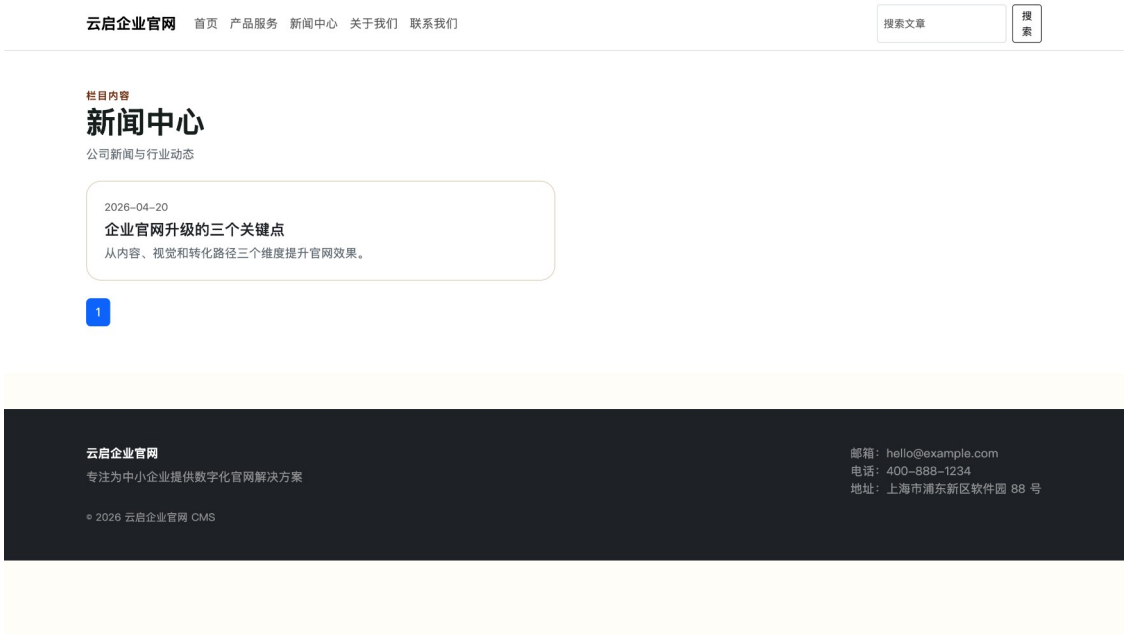

图 5-2 栏目文章列表页面

图 5-3 文章详情页面

文章详情路由的关键实现代码如下。

```
@bp.route('/article/<slug>')
def article_detail(slug):
    article = Article.query.filter_by(
        slug=slug, status='published').first_or_404()
    article.view_count += 1
    db.session.commit()
    related_articles = (
        Article.query.filter(
            Article.category_id == article.category_id,
            Article.id != article.id,
            Article.status == 'published')
        .order_by(Article.published_at.desc().nullslast())
        .limit(5).all()
    )
    return render_template('main/article_detail.html',
        article=article, related_articles=related_articles)
```

5.1.3 联系留言实现

联系留言页面对应 contact 路由，页面提供姓名、邮箱、电话和留言内容四个输入字段，所有字段均设置了 WTForms 验证规则。表单内嵌 CSRF 令牌防护跨站请求伪造，提

交成功后留言数据保存为 ContactMessage 模型记录。联系留言页面如图 5-4 所示。

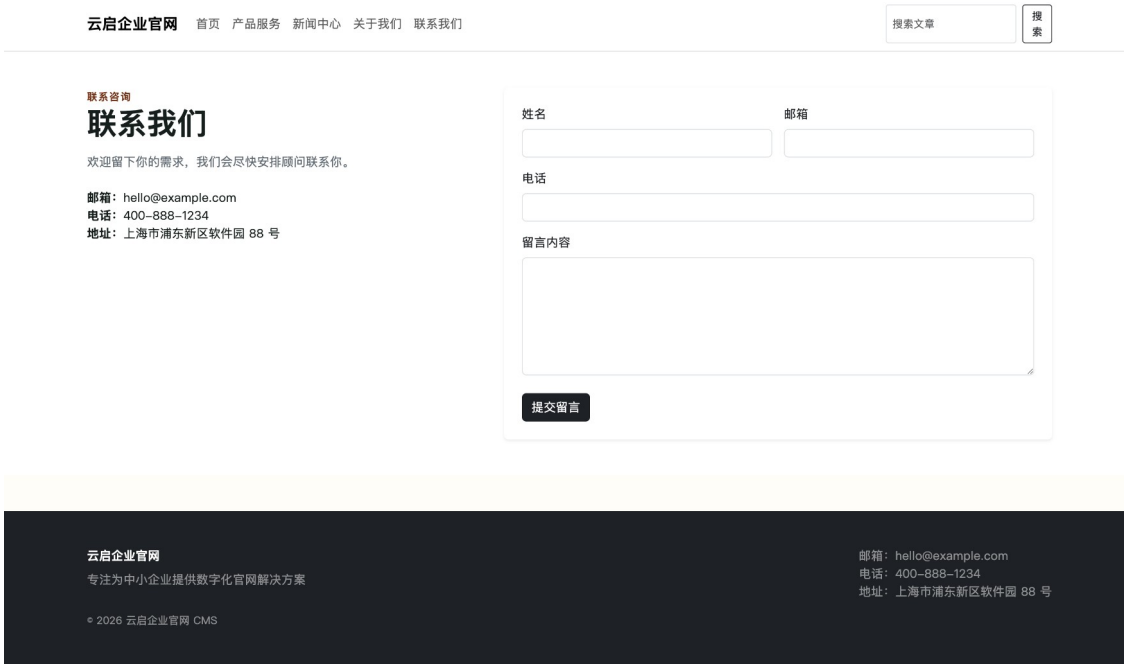

图 5-4 联系留言页面

联系表单提交的关键实现代码如下。

```
@bp.route('/contact', methods=['GET', 'POST'])
def contact():
    form = ContactForm()
    if form.validate_on_submit():
        message = ContactMessage(
            name=form.name.data.strip(),
            email=form.email.data.strip(),
            phone=(form.phone.data or '').strip(),
            message=form.message.data.strip())
        db.session.add(message)
        db.session.commit()
        flash('留言已提交，我们会尽快联系您。', 'success')
        return redirect(url_for('main.contact'))
    return render_template('main/contact.html', form=form)
```

5.2 后台功能模块实现

后台功能模块由 auth 蓝图和 admin 蓝图共同承载。auth 蓝图负责管理员登录与登出，admin 蓝图负责仪表盘、文章管理、栏目管理、单页管理、媒体管理、留言处理、用户管

理和站点设置等功能。所有后台路由都加了 Flask-Login 的 login_required 装饰器，关键操作还额外加了自定义的 permission_required 装饰器做权限校验。

5.2.1 管理员登录实现

管理员登录由 auth 蓝图的 login 路由处理，用户提交用户名和密码后，系统先用 WTForms 做非空校验和 CSRF 令牌验证，然后根据用户名查询 User 记录并调用 check_password 方法比对密码哈希值。验证没问题后调用 Flask-Login 的 login_user 函数创建会话，并更新 last_login_at 字段记录登录时间。管理员登录页面如图 5-5 所示。

图 5-5 管理员登录页面

登录路由的关键实现代码如下。

```
@bp.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('admin.dashboard'))
    form = LoginForm()
    if form.validate_on_submit():
```

```
        user = User.query.filter_by(
            username=form.username.data.strip()).first()
        if user and user.status \
                and user.check_password(form.password.data):
            login_user(user)
            user.last_login_at = utcnow_naive()
            db.session.commit()
            flash('登录成功。', 'success')
            next_url = request.args.get('next') \
                or url_for('admin.dashboard')
            return redirect(next_url)
        flash('用户名或密码错误。', 'danger')
    return render_template('auth/login.html', form=form)
```

5.2.2 后台仪表盘实现

后台仪表盘对应 admin 蓝图的 dashboard 路由，页面顶部展示文章总数、已发布文章数、栏目数、留言数和用户数五项统计数据，下方分别展示最近发布的五篇文章和最近收到的五条留言。仪表盘为管理员提供了系统运行状态的全局视图。后台仪表盘页面如图 5-6 所示。

图 5-6 后台仪表盘页面

仪表盘路由的关键实现代码如下。

```
@bp.route('/')
@login_required
@permission_required('dashboard.view')
def dashboard():
    stats = {
        'article_count': Article.query.count(),
        'published_count': Article.query.filter_by(
            status='published').count(),
        'category_count': Category.query.count(),
        'message_count': ContactMessage.query.count(),
        'user_count': User.query.count(),
    }
    latest_articles = Article.query.order_by(
        Article.created_at.desc()).limit(5).all()
    latest_messages = ContactMessage.query.order_by(
        ContactMessage.created_at.desc()).limit(5).all()
    return render_template('admin/dashboard.html',
        stats=stats, latest_articles=latest_articles,
        latest_messages=latest_messages)
```

5.2.3 文章管理实现

文章管理包含列表展示、新建、编辑和删除四个核心操作。文章列表页支持按发布状态和关键词组合筛选，查询结果用 paginate 方法做分页。新建文章时系统根据标题自动生成 URL 别名 slug，封面图片经扩展名白名单校验后以哈希文件名保存至 uploads 目录。文章状态支持草稿、已发布和已归档三种取值的流转。文章管理页面如图 5-7 所示。

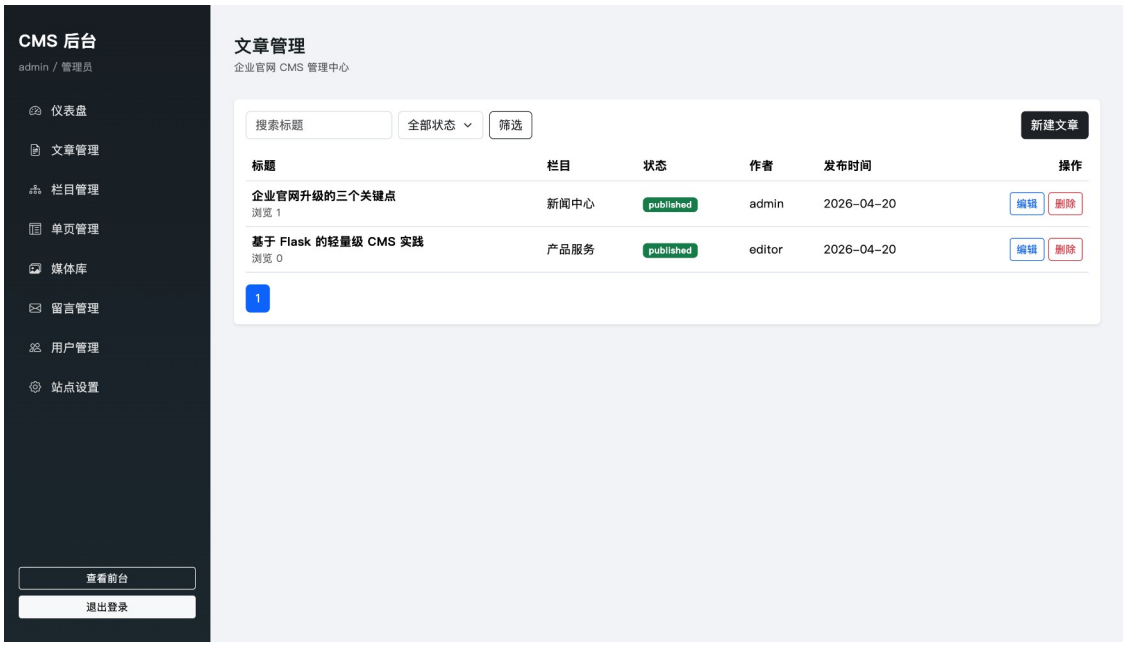

图 5-7 文章管理页面

文章列表路由支持按状态和关键词筛选，关键实现代码如下。

```
@bp.route('/articles')
@login_required
@permission_required('article.view')
def article_list():
    page = request.args.get('page', 1, type=int)
    status = request.args.get('status', '', type=str)
    keyword = request.args.get('q', '', type=str).strip()
    query = Article.query
    if status:
        query = query.filter_by(status=status)
    if keyword:
        query = query.filter(
            Article.title.ilike(f'%{keyword}%'))
    pagination = query.order_by(
        Article.is_top.desc(),
        Article.created_at.desc()
    ).paginate(page=page, per_page=10, error_out=False)
    return render_template('admin/article_list.html',
        pagination=pagination, keyword=keyword,
        status=status)
```

5.2.4 站点设置实现

站点设置功能对应 admin 蓝图的 settings 路由，全局参数以键值对方式管理。页面加载时从 SiteSetting 表读取所有键值对并填充到表单中，管理员修改保存后系统遍历表单字段逐一更新对应的数据库记录，若键不存在则自动创建新记录。站点设置页面如图 5-8 所示。

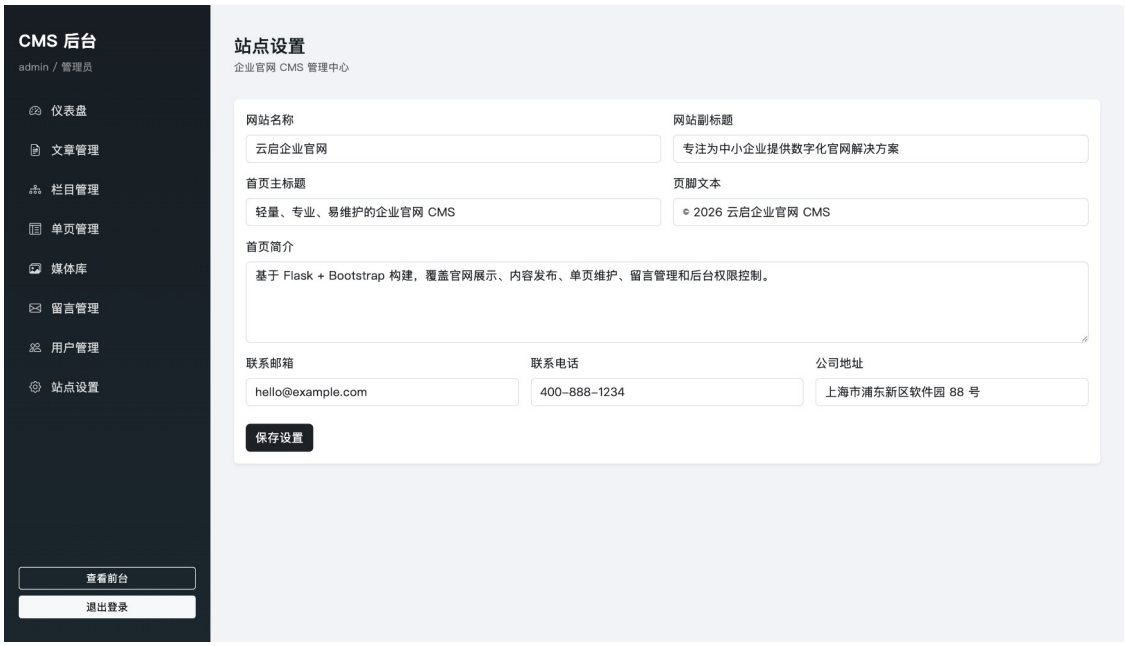

图 5-8 站点设置页面

站点设置路由的关键实现代码如下。

```
@bp.route('/settings', methods=['GET', 'POST'])
@login_required
@permission_required('setting.view')
def settings():
    key_map = {item.key: item
             for item in SiteSetting.query.all()}
    form = SiteSettingForm(data={
        key: key_map.get(key).value
        if key in key_map else ''
        for key in SiteSettingForm()._fields
        if key != 'csrf_token'})
    if form.validate_on_submit():
        for key, field in form._fields.items():
            if key in {'csrf_token', 'submit'}:
```

```
            continue
        setting = key_map.get(key)
        if setting is None:
            setting = SiteSetting(
                key=key, label=field.label.text,
                group_name='general')
            db.session.add(setting)
        setting.value = field.data or ''
    db.session.commit()
    flash('站点设置已保存。', 'success')
    return redirect(url_for('admin.settings'))
return render_template('admin/settings.html',
    form=form)
```

5.3 本章小结

本章分前台和后台两部分，展示了各主要页面的运行效果和对应的路由代码。前台部分包括首页展示、栏目文章浏览和联系留言，后台部分包括管理员登录、仪表盘、文章管理和站点设置。每个功能均以运行截图配合核心代码的方式呈现，体现了 Flask 蓝图机制、SQLAlchemy 查询构建和 Flask-Login 权限控制在实际业务场景中的应用。

6 系统测试

6.1 测试概述

系统测试用来验证功能是否完整、运行是否稳定，是交付前检查系统质量的必要步骤。企业官网 CMS 涉及前台页面展示、后台内容管理、用户身份认证、权限访问控制和数据持久化等多个相互关联的功能模块，各模块之间存在复杂的数据传递和状态依赖关系。为了在部署前确认系统是否达到了需求分析阶段提出的功能要求和性能指标，本章从测试概述、测试方法、功能测试用例设计以及测试结果分析四个方面，对系统进行全面系统的测试验证工作。

测试在本地开发环境下进行，硬件配置为 Intel Core i5 处理器、16GB 内存、512GB 固态硬盘，操作系统为 Windows 11。后端运行环境为 Python 3.12，Web 框架为 Flask 3.0，数据库采用 SQLite。测试浏览器为 Google Chrome 124，同时使用 Firefox 和 Edge 浏览器进行页面兼容性验证。测试数据库中预置了 3 个栏目分类、15 篇包含不同状态的测试文章、2 个具有不同角色权限的后台用户账号和 5 条留言记录，基本覆盖了系统日常运行的数据规模和业务场景多样性。

6.2 测试方法

6.2.1 功能测试方法

功能测试用的是黑盒方法，测试人员不看代码内部逻辑，仅根据需求规格设计测试输入并观察系统输出是否与预期一致。测试用例按前台访客和后台管理员两类角色分组

设计，模拟真实使用场景逐项验证各功能模块的正确性。页面层面在浏览器中实际操作来验证交互效果和数据展示，接口层面则观察页面响应状态和数据库记录变化来判断处理结果。

6.2.2 性能测试方法

性能测试用 Chrome 开发者工具的 Network 面板记录页面加载时间和静态资源传输耗时，Performance 面板用来观察页面渲染帧率和 JavaScript 执行效率。后端接口响应时间从 Flask 内置的请求处理日志中统计。性能达标标准为页面完整加载不超过 3 秒，后端接口响应不超过 500 毫秒，页面交互操作无明显卡顿现象。

6.3 功能测试

6.3.1 前台功能测试

前台功能测试覆盖首页展示、栏目浏览、文章详情、站内搜索和联系留言等访客常用功能模块，共设计 7 个测试用例。测试用例及结果如表 6-1 所示。

表 6-1 前台功能测试用例

测试编号	测试项	测试步骤	预期结果	测试结果
TC-01	首页访问	在浏览器中访问系统首页地址	页面正常加载，显示导航菜单、置顶文章和最新文章列表	通过
TC-02	栏目浏览	点击导航菜单中的新闻栏目链接	进入栏目页面，按时间倒序展示该栏目下已发布文章	通过
TC-03	文章详情查看	在栏目列表中点击某篇文章标题	进入文章详情页，显示完整正文、发布时间和	通过

			阅读次数	
TC-04	分页功能	在文章列表页点击第二页页码	页面跳转至第二页，显示对应区间的文章记录	通过
TC-05	站内搜索	在搜索框输入关键词 Flask 并提交	返回标题或正文中包含 Flask 的文章列表	通过
TC-06	联系留言提交	填写姓名、邮箱、电话和留言内容后提交	页面提示留言提交成功，数据库中新增一条留言记录	通过
TC-07	留言表单校验	不填写邮箱直接提交联系表单	页面提示邮箱字段不能为空，表单未被提交	通过

6.3.2 后台功能测试

后台功能测试覆盖管理员登录、文章增删改查、栏目管理、媒体上传和站点设置等管理功能模块，共设计 8 个测试用例。测试用例及结果如表 6-2 所示。

表 6-2 后台功能测试用例

测试编号	测试项	测试步骤	预期结果	测试结果
TC-08	管理员登录	输入正确的管理员用户名和密码并提交	登录成功，跳转至后台仪表盘页面	通过
TC-09	错误密码登录	输入正确用户名和错误密码并提交	页面提示用户名或密码错误，停留在登录页	通过
TC-10	未登录访问后台	未登录状态下直接访问后台管理地址	系统自动重定向至登录页面	通过
TC-11	新建文章	进入文章管理页，点击新建，填写标题和正文后保存	文章创建成功，列表中出现新记录且状态为草稿	通过
TC-12	编辑文章	选择一篇已有文章，修改标题后保存	修改成功，列表中文章标题更新为新值	通过
TC-13	删除文章	选择一篇文章点击删除并确认	文章从列表中移除，数据库中对应记录被删除	通过
TC-14	媒体文件上传	在媒体管理页面上传一张合法格式的图片文件	上传成功，媒体列表中出现新记录并显示缩略图	通过
TC-15	站点设置修改	修改网站名称和	保存成功，前台	通过

		联系电话后保存	页面导航栏和页脚同步显示新值

6.4 测试结果与分析

15 个功能测试用例全部跑完，实际结果和预期一致，没有出现页面崩溃、数据丢失或权限失效的情况。前台测试里，首页展示、栏目浏览、文章详情和站内搜索都能正确响应访客操作。联系留言的表单校验能拦截不合规输入，合法留言数据写入了数据库，后台管理界面可以查看和处理。后台测试里，登录认证可以区分合法管理员和未授权访问，未登录时直接访问后台地址会被重定向到登录页。文章的新增、编辑、状态切换和删除操作在数据库层面都正确反映了，数据一致性没有问题。

性能测试结果显示，前台首页完整加载时间约为 1.5 秒，栏目列表页和文章详情页加载时间均在 1 秒以内，后端接口平均响应时间约为 80 毫秒，各项指标均在设定的达标范围内。Chrome、Firefox 和 Edge 三个浏览器上页面的渲染效果和交互行为一致，移动端模拟器中响应式布局的适配也符合预期。由于系统采用 SQLite 数据库且测试数据量较小，当前性能表现良好，但在数据规模和并发访问量显著增长的场景下，仍需考虑将数据库迁移至 MySQL 或 PostgreSQL 并引入缓存优化方案。

从测试过程中发现的问题来看，测试初期文章列表页在筛选条件为空时未正确显示全部记录，排查后发现是查询条件拼接逻辑中对空字符串的判断存在遗漏，修复后经回归测试确认功能恢复正常。此外，在联系留言测试中发现电话号码字段未做格式校验，已在后续迭代中补充了正则表达式验证规则。整体上，各模块功能完整、运行稳定，安

全机制也正常工作，能满足中小型企业官网内容管理的基本使用需求。

6.5 本章小结

本章对企业官网 CMS 进行了系统化的功能测试和性能测试。功能测试按前台访客和后台管理员两类角色共设计 15 个测试用例，覆盖了首页展示、栏目浏览、文章详情、站内搜索、联系留言、管理员登录、文章增删改查、媒体上传和站点设置等核心功能，全部测试用例执行结果均与预期一致。性能测试显示，当前数据规模下页面加载速度和接口响应时间都在达标范围内，跨浏览器兼容性和响应式布局也没有明显问题。测试中发现的两个缺陷都已修复并通过回归验证。测试结果表明系统功能完整、运行稳定，为后续的总结与展望提供了事实依据。

7 总结与展望

7.1 工作总结

本文围绕中小型企业官网内容发布与后台维护的实际需求，完成了一套基于 Flask 微框架和 Bootstrap 5 前端框架的企业官网内容管理系统的设计与开发。论文从企业官网的信息展示、栏目组织、文章维护、媒体管理和访客留言等典型业务场景出发，依次完成了系统需求分析与可行性论证、总体架构设计与数据库建模、前后台功能开发实现以及系统测试验证四个阶段的工作，形成了从需求到交付的完整开发闭环。

设计上按 MVC 架构模式分层组织，Flask 蓝图机制把前台展示、用户认证和后台管理三块业务拆开，模块之间的耦合度因此降低，后续维护和扩展也更方便。数据库围绕角色、用户、文章、栏目、单页、媒体资源、站点设置和联系留言八个实体来设计。SQLAlchemy ORM 建立了各实体之间的数据模型和关联关系，前后台功能的数据读写都依赖这套模型。

功能实现上，前台模块做了首页品牌展示、栏目分类浏览、文章详情阅读与浏览量统计、站内关键词搜索和联系留言提交，这些都是面向访客的功能。后台模块覆盖了管理员登录与会话管理、仪表盘数据概览、文章管理（草稿→发布→归档三状态流转）、栏目层级维护、单页编辑、媒体上传、用户账号维护、站点全局参数配置等功能。系统在技术复杂度和运维成本上都控制得比较低，能支撑企业官网的内容更新和日常维护，和轻量级 CMS 的设计初衷一致。

安全方面，Flask-Login 管用户会话，Werkzeug 做密码哈希存储，Flask-WTF 防 CSRF，再加上自定义的角色访问控制装饰器，从用户认证到操作授权这条链路上的安全措施都有覆盖。功能测试和性能测试中，前台与后台共 15 个核心测试用例全部通过，页面加载速度和接口响应时间在预设标准范围内。系统目前具备教学演示和小规模部署的条件。

7.2 不足与展望

系统的核心功能已经完成，不过受开发周期和个人技术水平限制，和成熟的商业 CMS 产品比还有差距，主要在以下几个地方。内容编辑方面，文章正文目前是普通文本域输入，没有所见即所得的富文本编辑功能。图文混排和多媒体嵌入操作不够方便，版本历史回溯和定时发布等高级特性也还没有做。在搜索功能方面，站内检索目前用的是数据库 LIKE 模糊匹配，当文章数据量增长到较大规模时检索效率和相关性排序能力可能受到影响。在数据存储方面，默认使用的 SQLite 数据库属于嵌入式单文件数据库，虽然部署简便，但在高并发写入场景下存在锁争用问题，水平扩展能力有限。

后续改进可以从几个方向入手。内容编辑方面，可以集成 CKEditor 或 TinyMCE 等富文本编辑器，增强文章排版和多媒体嵌入能力。同时引入文章版本管理和定时发布队列，让内容发布流程更规范。性能方面，数据库可以从 SQLite 迁到 MySQL 或 PostgreSQL，获得更好的并发处理能力。首页推荐文章、栏目导航列表这类高频数据可以用 Redis 做缓存，减轻数据库压力。搜索功能可以考虑接入 Elasticsearch 替代数据库的 LIKE 模糊查询，

检索效率和准确性都会好一些。

部署运维方面，可以加上 Docker 容器化打包和 docker-compose 编排，用 GitHub Actions 或 Jenkins 做自动化 CI/CD。日志采集、异常监控和定期数据备份这些机制也需要补上，让系统在生产环境下能长期稳定运行。功能扩展方面，后续可以考虑多语言国际化、用户评论互动、文章标签体系和数据统计分析仪表盘等模块，让系统适用的场景更多一些。

从最终结果看，系统在功能覆盖和工程实现上达到了毕业设计阶段设定的目标，可以为中小型企业提供一套成本较低、维护方便的内容管理方案。后续如果在编辑体验、性能优化、安全加固和运维自动化几个方向上继续改进，系统还有进一步成长的空间。

参考文献：

[1]Miguel Grinberg.Flask Web Development:Developing Web Applications with Python[M].2nd ed.Sebastopol:O'Reilly Media,2018.

[2]李辉.Flask Web 开发实战:入门、进阶与原理解析[M].北京:机械工业出版社,2018.

[3]张海藩,牟永敏.软件工程导论(第 6 版)[M].北京:清华大学出版社,2013.

[4]陈嘉发,黄宇靖.Flask 框架在数据可视化的应用[J].福建电脑,2022(5):5.

[5]范路桥,段班祥,高洁.基于 Python+Flask+MySQL 的知宝问答系统[J].现代计算机,2022(11):6.

[6]任丹.基于 Flask 的计算机公共课智能题库设计[J].网络安全技术与应用,2021(2):2.

[7]赵北庚.基于 Flask 与爬虫技术的可视化深度学习数据标注系统[J].电子制作,2020(2):2.

[8]涂远杰,郑剑.基于 Flask 的博客网站设计与实现[J].电脑知识与技术,2020(3):3.

[9]张艺豪,盛丹红,翟丹丹.基于 Flask 和 Echarts 的光缆数据管理平台[J].电脑编程技巧与维护,2021(2):2.

[10]余晓帆,朱丽青.基于 Flask 框架的社交网站数据爬取及分析[J].微型电脑应用,2022(4):4.

[11]潘超,许烨.基于 Bootstrap 响应式的 Web 前端设计[J].无线互联科技,2022(2):2.

[12]谷顺利.基于 Bootstrap 框架响应式网页设计与实现[J].信息记录材料,2022(4):4.

[13]江家龙.基于 Bootstrap 的响应式餐饮网站设计与实现[J].现代信息科技,2021(5):5.

[14]陈明雪,周小丽,张受亭.面向 Web 前端开发的 Bootstrap 课程教学研究[J].电脑与电信,2022(4):104-107.

[15]直敏,高天哲,孙杨.响应式布局在网页设计中的应用[J].无线互联科技,2022(3):3.

[16]姚馨,黎明明,邓丽萍.响应式网页设计中媒体查询的应用和实现[J].现代信息科技,2023(5):5.

[17]郭蕊,赵元苏.Web 前端框架技术综述[J].北京工业职业技术学院学报,2021(4):4.

[18]周星岩,王俊淘.Web 主流前端开发框架的设计[J].电子技术与软件工程,2021(2):2.

[19]贺宗平,贺曦冉,秦新国.一种 Python ORM 框架性能测试分析方法研究[J].现代信息科技,2021(4):4.

[20]Ferraiolo D F,Sandhu R,Gavrila S,et al.Proposed NIST Standard for Role-Based Access Control[J].ACM Transactions on Information and System Security,2001,4(3):224-274.

[21]Pallets Projects.Flask Documentation(3.0.x)[EB/OL].https://flask.palletsprojects.com/,2024.

[22]Bootstrap Team.Bootstrap Documentation(v5.3)[EB/OL].https://getbootstrap.com/docs/5.3/,2024.

[23]Mike Bayer.SQLAlchemy Documentation[EB/OL].https://docs.sqlalchemy.org/,2024.

致谢

本论文的完成离不开许多人的帮助与支持，在此谨向他们致以诚挚的感谢。

首先，衷心感谢我的指导老师在毕业设计全过程中给予的悉心指导。从课题选定、技术方案确定到系统功能设计，再到论文结构调整和文字润色，老师始终以严谨的治学态度和丰富的工程经验为我指明方向，帮助我在遇到技术瓶颈时理清思路、找到解决方案。老师的专业素养和敬业精神让我受益匪浅，也为我今后的职业发展树立了榜样。

其次，感谢同学们在开发调试过程中给予的技术交流和经验分享。在系统功能实现和测试验证阶段，同学们热心地协助我进行页面兼容性测试、提出改进建议并帮助定位程序缺陷，使项目开发过程更加顺利高效。同窗数载的深厚情谊是我大学生活中最宝贵的收获之一。

最后，特别感谢我的家人在整个学业期间给予的无条件理解、支持和鼓励。正是家人的默默付出和殷切期望，给予了我克服困难、坚持完成学业的不竭动力。

本文的研究工作虽已告一段落，但仍存在诸多不足之处，我将在今后的工作和学习中不断弥补和提升。再次向所有关心和帮助过我的老师、同学和家人表示最衷心的感谢！